UCCD 2103
OPERATING SYSTEMS

OCT 2025

ASSIGNMENT
PART B

No.	Student ID	Name	Programme (CN/CS/CT/IA/IG)	E-mail
1				
2				
3				
4				

Assignment Part B (mutual exclusion and synchronization)

Grouping: Please follow the same group members as you have formed in Part A

Assessment type: Group (meaning each group member will get the same mark)

Objective: To expose students to POSIX C multi-threading, mutual exclusion and threads synchronization using semaphore.

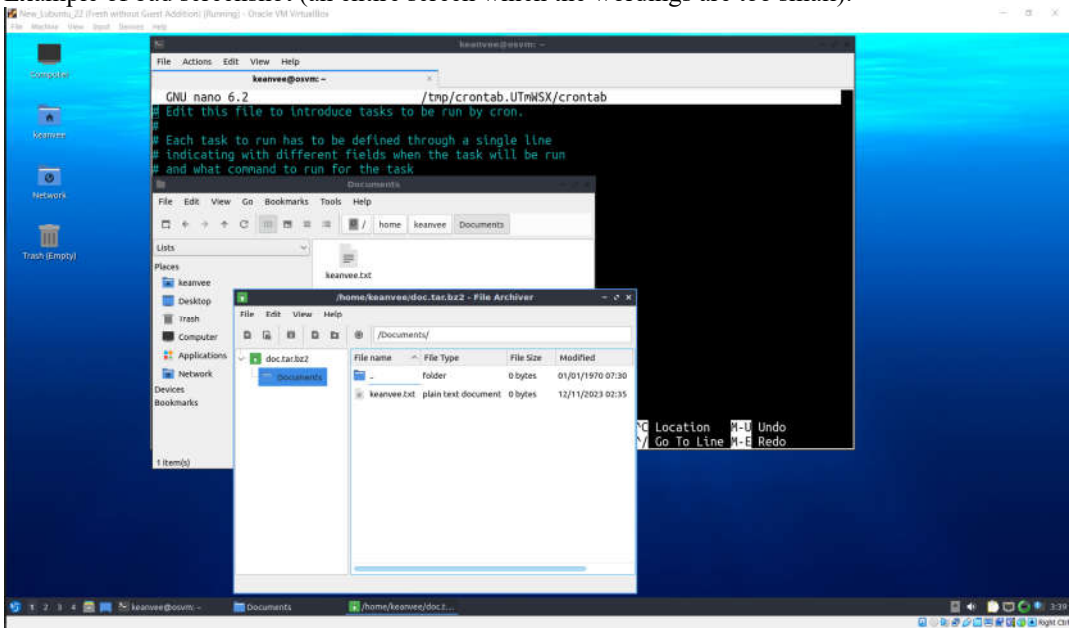
Due date/time: 11th December 2025 11.59 PM (link will be disabled automatically at 12.05 AM, 12th December 2025)

Submission:
Reports from all members should be combined into a single report and submit to WBLE. The link for Part B submission will be made available later. Only the group leader needs to submit one softcopy report, name the report as ***yourleadername.pdf***. Your submission is final (no resubmission is allowed). Thus, please make sure you submit the correct file. Use the report cover page provided. Assignment free riders should be reported and excluded from the report submission.

Report formatting:
Your assignment report must use the cover page given in this document. For the content, use font size 12.

Reminder:
All the screenshots (except the VM settings) in the report must contain / able to show your user name to **proof the originality of your work**. Notice that the screenshots included in this document show my user name "keanvee".
Any group reports that have the **same screenshots or usernames**, will be counted as **plagiarism** and be awarded 0 marks.
The content of all screenshots must be clear enough to be read. DO NOT take the screenshot of the entire screen.

Example of bad screenshot (an entire screen which the wordings are too small):



For Part A:
Inside the zip file that you submit to WBLE should contain two files:

- Your report in PDF:
  - Answer for Q1 and Q2
  - Appendix A: Content of `e-commerce-solution.c`
  - Appendix B: Content of `train-solution.c`
  - Appendix C: Virtual Machine specs that you use to test your programs.
- Source code files:
  - `e-commerce-solution.c`
  - `train-solution.c`

Tutorial: Multi-threading, race condition, and Semaphore

This tutorial is to guide you how to setup the gcc compiler, pthread library (for multithreading) and to demonstrate a simple multithreading process with shared resources.

1. *Multi-threading and race condition*
   a) This question is about multi-threading and race condition. It requires the following packages to be installed as **root** before you can attempt it. Thus, at your terminal, switch to root before executing the following steps:
      i. `gcc` C language compiler will be used.
      Command: `sudo apt install gcc`
      ii. `pthread` library is required to be installed. `pthread` is a POSIX Linux KLT threading library.
      Command to install: `sudo apt-get install libc6-dev`

   b) Switch back to your own user login. Download `multi-thread.c` source code from the WBLE.
      i. At your terminal, browse to the `Downloads` directory by running:
      `cd ~/Downloads`
      ii. To compile it into an executable file called `multi-thread`, enter:
      `gcc multi-thread.c -o multi-thread -lpthread -lrt`
      Note: You may get compilation warning, which you can ignore for this example.
      iii. To run it, enter: `./multi-thread`

      The program will create two concurrent threads that greet "Hello world!" together with their respective thread number. Both threads perform the same thing: (1) Identify their own thread number, and (2) Print their own thread number on the screen. Study the source codes to get familiar with the syntax of multi-threading using `pthread`.

2. *Threads Synchronization using Semaphore:*
   a) Download `race.c` source code from the WBLE. The process will create two threads and each thread will perform the calculation as shown in Figure 1 and print the final values of both variables after both threads have finished.

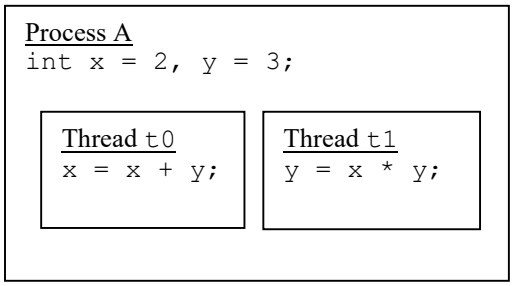


Figure 1

Compile it into an executable file called `race` (refer to step 1b above)and run it for 500 iterations by using the shell script provided, called `runner.sh`.
First, enable `runner.sh` to be executable: `chmod 755 runner.sh`
Then, run the script, enter: `./runner.sh race`
This script will automatically run the race program 500 iterations. Study the source codes and observe the output in the terminal: For 500 runs, you will notice some of the outputs are different from the rest. This is because race condition happened, i.e. if Thread `t0` runs first, the final output will be `x` = 5, `y` = 15. If Thread `t1` runs first, the final output will be `x` = 8, `y` = 6.

b) We can use Semaphore to control which thread to enter the critical section first. For example, we want thread `t1` to enter the critical section first. To learn how to use semaphore, you may download the `race-sem.c` sample source code as a tutorial. Compile it using: `gcc race-sem.c -o race-sem -lpthread -lrt` and run it using: `./race-sem`

c) Study how semaphore is implemented (declaration, initialization, `sem_wait`, `sem_post`, and `sem_destroy`). In that source code, we force thread `t0` to wait for thread `t1` so that thread `t1` will enter the Critical Section first every time we execute, and thus eliminate race condition all together.

d) Basically, a timeline table is used to trace every variable values per line of codes to understand why the solution is theoretically correct/how did you get such output in the terminal (as shown below) when you execute the solution source code. Timeline Table 1 is an example where **ONE of possible execution sequences** could have happened in a uniprocessor system that could lead to the result shown in the program output in Figure 2. Please take note how the values for `x`, `y`, and semaphore `s0` for each line of execution are traced.

Execution sequence	thread t0	thread t1	x	y	semaphore s0	Remark
0			2	3	0	Initial values. Let's say t0 run first
1	sem_wait(&s0)		2	3	-1	thread t0 is BLOCKED even though it run first
2		y=x*y	2	6	-1	
3		sem_post(&s0)	2	6	0	thread t0 is UNBLOCKED
4	x=x+y		8	6	0	Final value of x and y

Timeline table 1: Assume `t0` executes first. Hint: Even `t1` executes first, you still get the same output as below, every single time.

```
keanvee@osvm:~/Downloads/Assignment part B$ ./race-sem
The final value of x is 8
The final value of y is 6
```

Figure 2: Program output

Question: Threads synchronization and Semaphore question

1. Imagine a typical scenario that could happen in an e-commerce website where two customers are ordering a unit of Crucial 32GB DDR5 4800 SODIMM RAM from the same seller at the same time. The initial stock balance of that item is 2 units, before these customers ordered.

Figure 3: [2] For illustration.

The problem can be simulated by using two threads. Assume that:

- thread `t0` simulates customer 0 and orders 1 unit of Crucial 32GB.
- thread `t1` simulates customer 1 and orders 1 unit of Crucial 32GB.

Upon ordering the item, the e-commerce webpage checkout cart will first read the stock balance. Assume that the I/O latency consumes 0.1s to do so. Then it will check if there is stock available (balance is larger than 0). If that condition is true, it will deduct that stock and considered it as a completed order. Finally, the program will print out the stock balance after both customers had ordered.

A sample source code without semaphore is given. Download the file `e-commerce.c`, compile it (`gcc e-commerce.c -o e-commerce -lpthread -lrt`) and execute it (`./e-commerce`) to check if it is working. Then, execute `./runner.sh e-commerce` to loop the `e-commerce` program 500 times.

a) Write down your observation about the output of the program by describing what is wrong with the execution sequence. Please use a timeline table to investigate the issue. (10 marks)

b) Fix the issue by implementing semaphore into the `e-commerce.c` source code. Rename your solution file into `e-commerce-solution.c` and test the correctness by running your `e-commerce-solution` program 500

times (i.e. use the `runner.sh`). Show two variants of possible execution sequences by using a timeline table for each variant. (20 marks)
Hint: An excerpt of correct sample output is shown in the Figure 4. For this particular problem, there is no race condition if you manage to solve it correctly.

```
keanvee@osvm:~/Downloads$ ./runner.sh e-commerce-solution
Run no. 0 Final stock balance: 0
Run no. 1 Final stock balance: 0
Run no. 2 Final stock balance: 0
Run no. 3 Final stock balance: 0
Run no. 4 Final stock balance: 0
Run no. 5 Final stock balance: 0
Run no. 6 Final stock balance: 0
Run no. 7 Final stock balance: 0
Run no. 8 Final stock balance: 0
Run no. 9 Final stock balance: 0
Run no. 10 Final stock balance: 0
Run no. 11 Final stock balance: 0
Run no. 12 Final stock balance: 0
Run no. 13 Final stock balance: 0
Run no. 14 Final stock balance: 0
Run no. 15 Final stock balance: 0
Run no. 16 Final stock balance: 0
Run no. 17 Final stock balance: 0
Run no. 18 Final stock balance: 0
Run no. 19 Final stock balance: 0
Run no. 20 Final stock balance: 0
Run no. 21 Final stock balance: 0
Run no. 22 Final stock balance: 0
Run no. 23 Final stock balance: 0
```

Figure 4

2. Figure 5 shows a gated road intersection where a railway line crosses a road. Before the train is crossing the railway, the railway gate will be closed. The road traffic must stop if the railway gate is closed. The road traffic can resume its journey once the train finished crossing and then railway gate is opened.

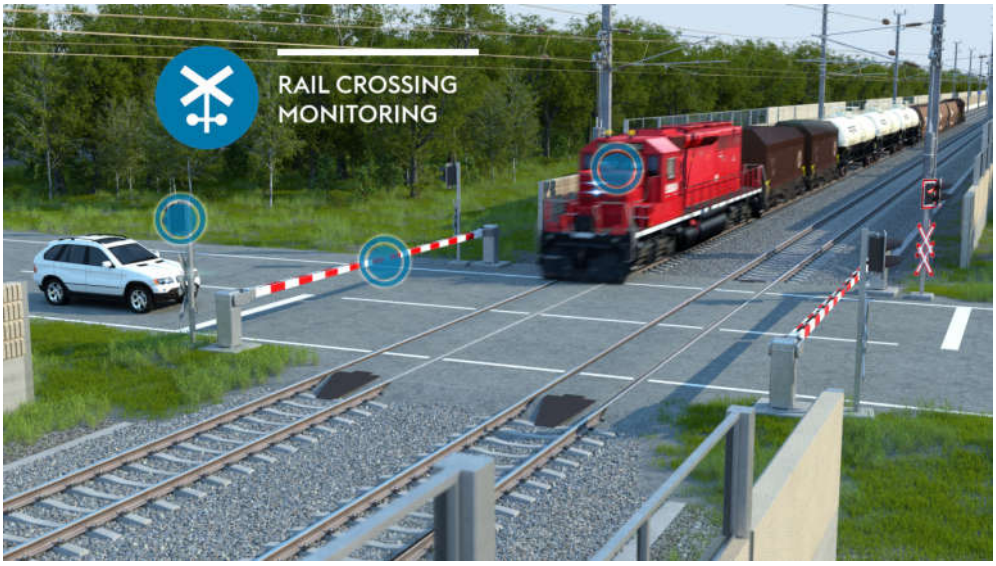


Figure 5 [3]

The problem can be simulated by using two threads. Assume that

- thread `train` simulates the train. The train uses 1 second to cross the road and a new train will arrive every 5 seconds.
- thread `traffic` simulates the car traffic. A 1 second sleep is added into this thread to prevent the infinite while loop from running too fast.

A sample source code without semaphore is given: Download file `train.c`, compile it (`gcc train.c -o train -lpthread -lrt`) and execute it (`./train`).

a) Write down your observation about the output of the `train` program by describing what is wrong with the execution sequence. Draw the timeline table to assist your explanation. (10 marks)

b) Fix the issue by implementing semaphore into the `train.c` source code. Rename your solution file into `train-solution.c` and test the correctness by running your solution multiple times. (20 marks)
Hint: A correct sample output is shown in the Figure 6. Please take note that **your output sequence could be different than Figure 6**, but still fulfilling the requirement of the problem.

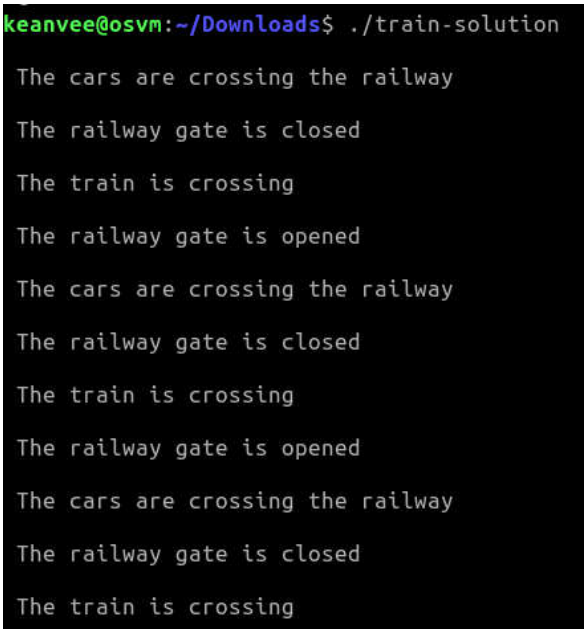


Figure 6

Report writing and tidiness (10 marks)

References:

[1] How to use POSIX semaphores in C language2018. [ONLINE] https://www.geeksforgeeks.org/use-posix-semaphores-c/. [Accessed 01 November 2025].

[2] CRUCIAL DDR5. [ONLINE] https://shopee.com.my/CRUCIAL-DDR5-4800MHZ-RAM-DESKTOP-PC-(UDIMM)-LAPTOP-NOTEBOOK-(SODIMM)-MEMORY-RAM-8GB-16GB-32GB-i.92746975.54001221948?extraParams=%7B%22display_model_id%22%3A218775637082%2C%22model_selection_logic%22%3A3%7D&sp_atk=6d1df262-f070-414c-9fe4-a398ac853dd5&xptdk=6d1df262-f070-414c-9fe4-a398ac853dd5

[3] RXM Rail Crossing Monitoring [ONLINE] https://www.voestalpine.com/signaling/en/products/Rail-Crossing-Monitoring/